

CICLO SUPERIOR DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA (DAM)

TRABAJO DE FIN DE GRADO SUPERIOR — MEMORIA TÉCNICA

CineClip

Aplicación multiplataforma de adivinanza de películas por fotograma



SERGIO GONZÁLEZ
INNOVATIONS

Alumno/a	Sergio González Chacón
Curso académico	2025 – 2026
Fecha de entrega	Mayo 2026

Índice de contenidos

1. Introducción

- 1.1. Motivación y contexto
- 1.2. Objetivos del proyecto
- 1.3. Tecnologías utilizadas
- 1.4. Estructura del documento

2. Estudio de Viabilidad del Sistema (EVS)

- 2.1. Objetivos del EVS
- 2.2. Descripción general del sistema
- 2.3. Diagrama de contexto
- 2.4. Situación actual
- 2.5. Catálogo de requisitos
- 2.6. Alternativas de construcción
- 2.7. Alternativa elegida y justificación
- 2.8. Aviso legal sobre propiedad intelectual
- 2.9. Planificación del proyecto

3. Análisis del Sistema de Información (ASI)

- 3.1. Entorno tecnológico
- 3.2. Catálogo de usuarios
- 3.3. Casos de uso
- 3.4. Modelo de datos conceptual

4. Diseño técnico

- 4.1. Arquitectura del sistema
- 4.2. Estructura de pantallas
- 4.3. Flujo de una partida
- 4.4. Campo de respuesta con autocompletado
- 4.5. Gestión del estado
- 4.6. Despliegue

5. Plan de pruebas

6. Conclusiones y trabajo futuro

7. Bibliografía

Anexo I. Modelo de Datos y Plan de Pruebas Completo

1. Introducción

1.1. Motivación y contexto

Este proyecto nace de una idea bastante sencilla: me gustan las películas y me gustan ese tipo de juegos de identificación visual que circulan por internet. Ya existen varias webs que hacen algo parecido, como Framed o Moviedle, pero tienen dos limitaciones que siempre me han molestado. La primera es que están pensadas solo para web, sin soporte real en móvil. La segunda es que te limitan a una partida al día, lo que corta bastante el juego cuando tienes ganas de seguir.

A partir de esa frustración surgió CineClip: una aplicación multiplataforma (Android y Web) que permite jugar sin límite de partidas, con imágenes de fondo (backdrops) obtenidas en tiempo real de la API pública de The Movie Database (TMDB), y con un ranking global de puntuaciones almacenado en Firebase.

Desde el punto de vista académico, el proyecto permite trabajar con tecnologías del plan de estudios del ciclo DAM que no siempre se ven en profundidad durante el curso: consumo de APIs REST, gestión de estado en aplicaciones móviles, servicios en la nube y despliegue multiplataforma desde una base de código única.

1.2. Objetivos del proyecto

El objetivo principal es desarrollar una aplicación funcional que permita al usuario adivinar películas a partir de un backdrop, con hasta 5 intentos por partida, backdrops progresivos y pistas de metadatos tras cada fallo. Más en concreto:

- Construir una aplicación multiplataforma (Android y Web) con React Native y Expo.
- Integrar la API de TMDB para obtener metadatos cinematográficos y backdrops en tiempo real, con contenido en español.
- Implementar backdrops progresivos: con cada fallo se muestra un backdrop diferente, de más ambiguo a más reconocible.
- Implementar un campo de respuesta con autocompletado usando el endpoint de búsqueda de TMDB.
- Implementar un sistema de ranking global persistente usando Firebase Firestore.
- Diseñar una interfaz sencilla e intuitiva, sin necesidad de manual o tutorial previo.

1.3. Tecnologías utilizadas

Tecnología	Versión	Uso en el proyecto
React Native + Expo	SDK 54	Framework principal de desarrollo multiplataforma.
TypeScript	5.x	Lenguaje de implementación. Tipado estático.
Firebase Firestore	Plan Spark (free)	Base de datos para ranking y jugadores.
TMDB API	v3 (REST)	Fuente de metadatos y backdrops en español. 3 endpoints por partida.
Navegación manual (App.tsx)	—	Gestión de pantallas mediante estado (useState) en App.tsx. Sin librería de navegación externa.
AsyncStorage	—	Persistencia local de la sesión activa.
Jest + RNTL	Jest 29	Framework de pruebas unitarias y de integración.
Firebase Authentication	Plan Spark (free)	Autenticación de usuarios con email y contraseña.
GitHub Pages	Free	Hosting de la versión web. Despliegue manual desde la rama gh-pages.
EAS Build (Expo)	Free	Generación del APK de Android en la nube.
react-native-safe-area-context	—	Gestión de zonas seguras en Android (barra de navegación) e iOS (notch).
@expo-google-fonts/bebas-neue	—	Tipografía Bebas Neue para títulos estilo Hollywood.

1.4. Estructura del documento

La memoria sigue la estructura clásica de un proyecto DAM: introducción, EVS, ASI, diseño técnico, plan de pruebas, conclusiones y anexos. El Anexo I recoge el modelo de datos completo con los campos reales verificados de la API de TMDB y el plan de pruebas con los 48 casos detallados.

2. Estudio de Viabilidad del Sistema (EVS)

El Estudio de Viabilidad del Sistema analiza si el proyecto es técnica y operativamente viable dentro del tiempo y los recursos disponibles. La conclusión es clara desde el principio: el proyecto es viable si se utiliza una tecnología conocida y se define bien el alcance del MVP.

2.1. Objetivos del EVS

- Evaluar las alternativas tecnológicas disponibles para el desarrollo multiplataforma.
- Definir el alcance del MVP y las funcionalidades que quedan fuera de esta versión.
- Identificar los principales riesgos y establecer medidas de contingencia.
- Establecer una planificación realista para el desarrollo del proyecto.

2.2. Descripción general del sistema

CineClip es una aplicación de entretenimiento. El jugador ve un backdrop de una película y tiene que adivinar su título usando un campo de búsqueda con autocompletado. Dispone de 5 intentos; con cada fallo el sistema muestra un backdrop diferente (progresivamente más reconocible) y desvela una pista adicional: año de estreno (fallo 1), director (fallo 2), género (fallo 3) y reparto principal (fallo 4). Si acierta antes de agotar los intentos, se registra la puntuación. Si falla las cinco veces, se muestra la respuesta correcta.

La aplicación funciona en Android y Web desde una única base de código. Los metadatos y backdrops se obtienen en tiempo real desde la API de TMDB en español, y el ranking global se almacena en Firebase Firestore.

2.3. Diagrama de contexto

Entidad externa	Datos que envía al sistema	Datos que recibe del sistema
Jugador	Selección de título (autocompletado)	Backdrop progresivo, feedback, pistas de metadatos, puntuación
API TMDB	Metadatos en español, backdrops, resultados de búsqueda	3 peticiones: /movie/{id}, /credits, /images
Firebase / BD	Ranking global, confirmación de alias	Registro de puntuación y alias
Administrador	Gestión y moderación	Reportes de errores, estadísticas

2.4. Situación actual

Existen varias aplicaciones similares en el mercado. Las más conocidas son Framed y Moviedle. Todas comparten el mismo patrón: una partida al día por usuario, diseño exclusivamente web y sin contenido en español. La principal ventaja diferencial de CineClip es la combinación de factores que ninguna ofrece a la vez: disponibilidad en móvil nativo, partidas ilimitadas, ranking global persistente, contenido en español y backdrops progresivos.

2.5. Catálogo de requisitos

2.5.1. Requisitos Funcionales

Código	Descripción
RF01	El sistema debe permitir al usuario registrarse con un alias y contraseña, o iniciar sesión si ya tiene cuenta, antes de acceder al juego. La autenticación se gestiona mediante Firebase Authentication.
RF02	El sistema debe mostrar un backdrop de una película aleatoria al iniciar cada partida, obtenido del endpoint /images de TMDB.
RF03	El usuario debe poder introducir el título mediante un campo de búsqueda con autocompletado que consulta el endpoint /search/movie de TMDB en español.
RF04	El sistema debe validar la respuesta comparando el ID de TMDB de la película seleccionada con el ID de la película activa. No se requiere normalización de texto.
RF05	El sistema debe limitar a 5 el número de intentos por partida.
RF06	Con cada fallo el sistema muestra un backdrop diferente (progresivamente más reconocible) y desvela una pista: año (fallo 1), director (fallo 2), género (fallo 3), reparto (fallo 4).
RF07	Al acertar o agotar intentos, el sistema mostrará la respuesta correcta y los metadatos completos de la película.
RF08	El sistema debe registrar y actualizar la puntuación acumulada del jugador tras cada partida.
RF09	El sistema debe mostrar un ranking global con las puntuaciones más altas almacenadas en Firebase.

2.5.2. Requisitos No Funcionales

Código	Categoría	Descripción
RNF01	Rendimiento	Los backdrops deben cargarse en menos de 2 segundos en conexión móvil estándar.
RNF02	Rendimiento	La aplicación debe funcionar de forma fluida en Android y Web.
RNF03	Seguridad	Todas las comunicaciones con TMDB y Firebase se realizarán mediante HTTPS.
RNF04	Seguridad	Las puntuaciones se almacenarán con reglas de acceso adecuadas en Firebase.
RNF05	Usabilidad	La interfaz debe ser usable sin manual ni tutorial previo.
RNF06	Usabilidad	Cualquier acción principal debe completarse con un máximo de 3 interacciones.
RNF07	Compatibilidad	Debe funcionar en Android 8+ y navegadores modernos.
RNF08	Compatibilidad	El diseño debe adaptarse automáticamente a móvil, tablet y escritorio.

RNF09	Mantenibilidad	El código debe organizarse por módulos para facilitar ampliaciones futuras.
RNF10	Accesibilidad	La interfaz debe cumplir requisitos básicos de contraste y tamaño de fuente.

2.6. Alternativas de construcción

2.6.1. Alternativa A — Flutter

Flutter es el framework de Google para desarrollo multiplataforma nativo. Usa Dart como lenguaje y tiene su propio motor de renderizado, lo que le da un rendimiento visual excelente.

Ventajas	Desventajas
Rendimiento nativo en todas las plataformas.	Requiere aprender Dart, que no está en el plan de estudios del ciclo.
Interfaces fluidas y animaciones de alta calidad.	Riesgo elevado de no cumplir los plazos del proyecto.
Gran comunidad y documentación extensa.	Curva de aprendizaje demasiado alta para un proyecto académico acotado.

Conclusión:

Flutter queda descartado. Requiere aprender Dart desde cero, algo inviable dentro del plazo disponible.

2.6.2. Alternativa B — React Native + Expo (Seleccionada)

Ventajas	Desventajas
Compatible con conocimientos previos en JavaScript y TypeScript.	Las animaciones complejas son menos fluidas que en Flutter.
Expo simplifica el despliegue en Android y Web.	Pueden aparecer incompatibilidades entre módulos nativos de terceros.
SDKs oficiales para Firebase y TMDB disponibles en npm.	La versión web de Expo no siempre tiene paridad total con las nativas.

Conclusión:

React Native con Expo es la opción elegida. Es la tecnología con la que más familiaridad tengo y la que permite cumplir los plazos con el nivel de riesgo más manejable.

2.7. Alternativa elegida y justificación

Se adopta React Native con Expo como framework principal, TypeScript como lenguaje de implementación, Firebase como backend y la API de TMDB como fuente de datos cinematográficos en español. La decisión se basa en cuatro criterios: conocimiento previo, despliegue multiplataforma real, integración sencilla con los servicios externos, y facilidad de distribución tanto en web (GitHub Pages) como en Android (APK generado con EAS Build).

2.8. Aviso legal sobre propiedad intelectual

CineClip es un proyecto con fines estrictamente educativos y sin ánimo de lucro. El uso de backdrops e imágenes cinematográficas se ampara en el derecho de cita para fines docentes (artículo 32 de la Ley de Propiedad Intelectual española). Todos los derechos sobre las películas, imágenes y metadatos pertenecen a sus respectivos propietarios. Los datos se obtienen a través de la API pública de TMDB, destinada exactamente a este tipo de uso no comercial.

2.9. Planificación del proyecto

El proyecto se divide en dos fases. La primera, de documentación, cubre de noviembre de 2025 a finales de febrero de 2026. La segunda, de desarrollo, va de marzo a mayo de 2026. Dedicación estimada: 8-10 horas semanales, total aproximado 311 horas.

ID	Tarea	Inicio	Fin	Horas
FASE 1 — Documentación (completada)				
T01	EVS — Objetivos y descripción	01/11/25	28/11/25	22 h
T02	Catálogo de requisitos	15/11/25	19/12/25	18 h
T03	Catálogo de usuarios	01/12/25	12/12/25	8 h
T04	Alternativas y solución propuesta	10/12/25	02/01/26	14 h
T05	Diagrama de casos de uso (UML)	05/01/26	30/01/26	28 h
T06	Modelo de datos conceptual	20/01/26	14/02/26	20 h
T07	Plan de pruebas	01/02/26	21/02/26	16 h
T08	Revisión documentación	17/02/26	28/02/26	10 h
FASE 2 — Desarrollo (en curso)				
T09	Config. entorno (Expo + Firebase)	02/03/26	13/03/26	16 h
T10	Pantallas base e interfaz	09/03/26	11/04/26	35 h
T11	Integración API TMDB (3 endpoints + idioma)	23/03/26	18/04/26	28 h
T12	Lógica del juego + backdrops progresivos	14/04/26	30/04/26	32 h
T13	Firebase (ranking + sesión)	24/03/26	30/04/26	24 h
T14	Pruebas y corrección de errores	02/05/26	15/05/26	20 h
T15	Memoria final y entrega	05/05/26	22/05/26	18 h
TOTAL ESTIMADO				311 h

3. Análisis del Sistema de Información (ASI)

El ASI profundiza en el análisis técnico: cómo se organiza la información, qué hace cada parte de la aplicación y cómo interactúan los actores con el sistema.

3.1. Entorno tecnológico

CineClip usa una arquitectura de cliente ligero orientada a servicios. No tiene backend propio: toda la lógica reside en el cliente (React Native), los datos cinematográficos vienen de TMDB en español y la persistencia se delega en Firebase.

Capa	Tecnología	Función
Frontend	React Native + Expo SDK 54	Interfaz y lógica de negocio. Genera builds para Android y Web.
Lenguaje	TypeScript 5.x	Tipado estático. Reduce errores en tiempo de ejecución.
Base de datos	Firebase Firestore	Ranking global y datos de jugadores. Plan gratuito Spark.
API externa	TMDB API v3	Metadatos en español y backdrops. 3 endpoints por partida.
Navegación	Navegación manual (App.tsx)	Gestión de pantallas mediante estado React (useState) en App.tsx.
Persistencia local	AsyncStorage	Estado de la sesión activa en el dispositivo.

3.2. Catálogo de usuarios

Actor	Descripción	Funciones	Acceso
Jugador	Usuario final. Interacción lúdica sin privilegios de administración.	Jugar partidas, usar autocompletado, consultar ranking.	App móvil y versión web.
Administrador	Responsable de supervisar el sistema. No juega.	Moderar ranking, supervisar Firestore, gestionar API Key de TMDB.	Firebase Console y portal TMDB.

3.3. Casos de uso

Se han identificado 13 casos de uso distribuidos entre los dos actores del sistema.

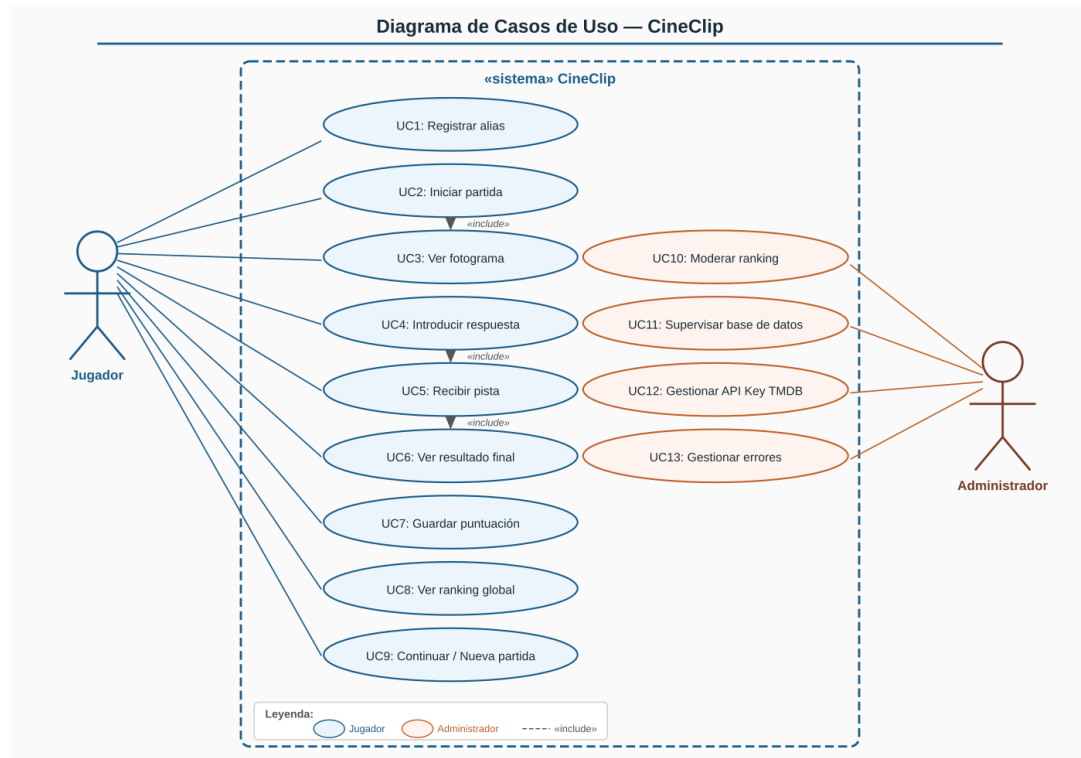


Figura 1. Diagrama de Casos de Uso — CineClip

UC	Nombre	Actor	Descripción breve
UC1	Registrar alias	Jugador	Registrar cuenta nueva o iniciar sesión con alias y contraseña mediante Firebase Authentication.
UC2	Iniciar partida	Jugador	Cargar película aleatoria y sus backdrops desde TMDB.
UC3	Ver backdrop	Jugador	Mostrar el backdrop activo de la película.
UC4	Buscar y seleccionar título	Jugador	Autocompletar título con /search/movie y validar por ID.
UC5	Recibir pista + nuevo backdrop	Jugador	Mostrar siguiente backdrop y pista de metadatos tras fallo.
UC6	Ver resultado final	Jugador	Mostrar resultado al acertar o agotar intentos.
UC7	Guardar puntuación	Jugador	Registrar puntuación en Firestore.
UC8	Ver ranking global	Jugador	Consultar clasificación desde Firebase.

UC9	Continuar / Nueva partida	Jugador	Reiniciar estado e iniciar nueva película.
UC10	Moderar ranking	Administrador	Eliminar registros ofensivos o fraudulentos.
UC11	Supervisar base de datos	Administrador	Revisar métricas y estado de Firestore.
UC12	Gestionar API Key TMDb	Administrador	Supervisar consumo y renovar si es necesario.
UC13	Gestionar errores	Administrador	Resolver incidencias de persistencia.

3.4. Modelo de datos conceptual

El modelo distingue entre entidades persistentes en Firebase y entidades externas de TMDb. Para obtener todos los datos de una partida se necesitan tres peticiones a la API, todas en español.

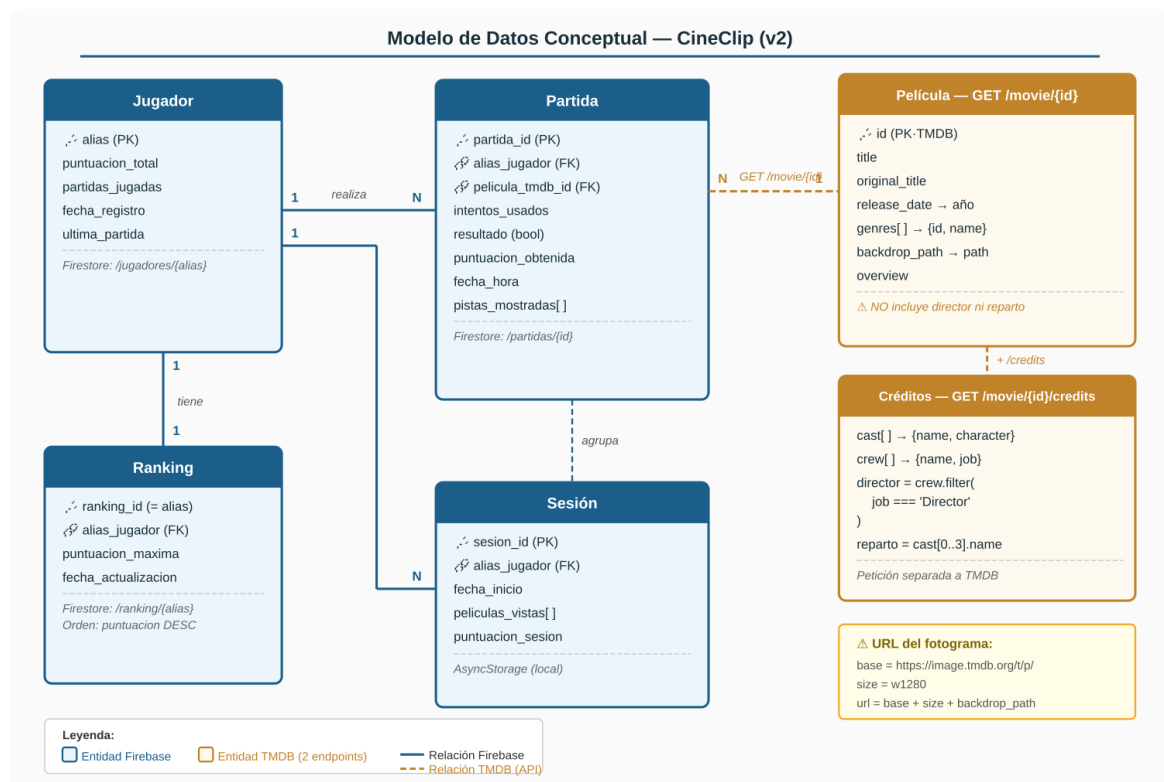


Figura 2. Diagrama Entidad-Relación — CineClip (campos TMDb verificados)

Endpoints TMDb por partida (language=es-ES en todas):

1. GET /movie/{id} → título en español, año, género
2. GET /movie/{id}/credits → director (crew[job==='Director']), reparto (cast[0..3])
3. GET /movie/{id}/images → array backdrops[], se eligen hasta 5 de forma progresiva

Colección Firestore	ID documento	Campos principales
/jugadores/{alias}	alias	puntuacion_total, partidas_jugadas, fecha_registro, ultima_partida
/partidas/{id} (reservado para versiones futuras)	UUID	alias_jugador, pelicula_tmdb_id, intentos_usados, resultado, puntuacion_obtenida, fecha_hora, pistas_mostradas[]
/ranking/{alias}	alias	alias_jugador, puntuacion_maxima, fecha_actualizacion

4. Diseño técnico

4.1. Arquitectura del sistema

CineClip sigue una arquitectura de cliente ligero orientada a servicios, implementada mediante Clean Architecture y los principios SOLID. No hay backend propio: toda la lógica reside en el cliente, pero organizada en capas con dependencias unidireccionales que facilitan el mantenimiento y las pruebas.

La regla fundamental es que las capas internas no conocen a las externas. El dominio no sabe nada de React Native, Firebase ni TMDB. La presentación no llama directamente a las APIs externas. Las dependencias siempre apuntan hacia el centro.

4.1.1. Capas de la arquitectura

Capa	Carpeta	Responsabilidad
Domain	src/domain/	Núcleo de la aplicación. Entidades, interfaces de repositorio y casos de uso. TypeScript puro, sin dependencias externas.
Data	src/data/	Implementaciones concretas de los repositorios. TmdbMovieRepository, FirebaseRankingRepository y FirebaseAuthRepository. Conoce las APIs externas.
Presentation	src/presentation/	Pantallas, componentes y hooks de React Native. Depende solo de las interfaces del dominio, nunca de las implementaciones concretas.
Infrastructure	src/infrastructure/	Configuración, constantes y cliente HTTP. Centraliza las claves y parámetros globales como language=es-ES.

4.1.2. Estructura de carpetas

Estructura del proyecto:

```
cineclip/
├── src/
│   ├── domain/
│   │   ├── entities/      # Movie, Player, Game, Score
│   │   ├── interfaces/
│   │   │   ├── repositories/ # IAuthRepository, IMovieRepository, IRankingRepository
│   │   │   └── usecases/
│   │   │       ├── auth/      # ILoginUser, IRegisterUser
│   │   │       ├── IGetRanking.ts
│   │   │       ├── ISaveSessionScore.ts
│   │   │       ├── IStartGame.ts
│   │   │       └── ISubmitAnswer.ts
│   │   └── usecases/
│   │       └── auth/      # LoginUser, RegisterUser
```

```

| | | | — GetRanking.ts
| | | | — SaveSessionScore.ts
| | | | — StartGame.ts
| | | | — SubmitAnswer.ts
| | | — data/
| | | | — tmdb/      # TmdbMovieRepository (implementa MovieRepository)
| | | | — firebase/  # FirebaseRankingRepository, FirebaseAuthRepository
| | | — di/
| | | | — container.ts # Ensambla repositorios y casos de uso manualmente
| | | | — types.ts    # Tokens de inyección (TYPES.MovieRepository, etc.)
| | | — presentation/ # Patrón MVVM: views = View, hooks = ViewModel
| | | | — views/      # AliasView, GameView, ResultView, RankingView
| | | | — components/ # BackdropImage, SearchInput, HintList, ScoreCard
| | | | — viewModel/   # useGame, useSearch, useRanking (ViewModels)
| | | — infrastructure/
| | | | — config/      # firebaseConfig.ts, tmdbConfig.ts
| | | | — http/        # Cliente fetch con language=es-ES y API Key
| — App.tsx           # Punto de entrada, solo navegación
| — ...

```

La capa de presentación sigue el patrón MVVM adaptado a React Native: las vistas (views/) solo renderizan y delegan acciones, los hooks (viewModel/) actúan como ViewModels exponiendo estado y funciones sin tomar decisiones de negocio. La inyección de dependencias se realiza de forma manual en di/container.ts, sin librerías externas, lo que facilita la comprensión y el mantenimiento del código.

4.1.3. Principios SOLID aplicados

S	Principio	Aplicación en CineClip
S	Single Responsibility	Cada clase tiene una única razón para cambiar. TmdbMovieRepository solo gestiona llamadas a TMDB. GameView solo gestiona la UI del juego.
O	Open/Closed	Las pantallas están abiertas a extensión pero cerradas a modificación. Si se añade un nuevo repositorio de datos, no se toca la presentación.
L	Liskov Substitution	TmdbMovieRepository puede sustituirse por cualquier otra implementación de MovieRepository sin que la presentación lo note.
I	Interface Segregation	MovieRepository solo tiene métodos de películas. RankingRepository solo tiene métodos de ranking. Interfaces pequeñas y específicas.
D	Dependency Inversion	La presentación depende de la interfaz MovieRepository, nunca de TmdbMovieRepository directamente. Las dependencias apuntan hacia el dominio.

4.1.4. Flujo de dependencias

El flujo de dependencias en una llamada típica, por ejemplo al validar la respuesta del jugador, sigue siempre la misma dirección:

Flujo de una validación:

GameView (presentation/views)

- useGame viewModel (presentation/viewModel)
- SubmitAnswer usecase (domain/usecases)
- MovieRepository interface (domain/repositories)
- ← TmdbMovieRepository (data/tmdb) [inyectado]

GameView nunca importa TmdbMovieRepository directamente.

SubmitAnswer nunca importa nada de React Native ni de Firebase.

4.2. Estructura de pantallas

Pantalla	Componente	Contenido
Alias / Bienvenida	AliasView	Campo para introducir el alias. Solo aparece la primera vez o al resetear sesión.
Juego	GameView	Backdrop activo, campo de búsqueda con autocompletado, contador de intentos y pistas acumulativas.
Resultado	ResultView	Resultado de la partida, metadatos completos en español y botón de nueva partida.
Ranking	RankingView	Lista de jugadores ordenada por puntuación descendente, obtenida de Firestore.

4.3. Flujo de una partida

Al iniciar una partida se lanzan en paralelo tres peticiones a TMDB (todas con language=es-ES):

- GET /movie/{id}?language=es-ES → título en español, año (de release_date), género (genres[0].name).
- GET /movie/{id}/credits?language=es-ES → director (crew.find(job==='Director').name), reparto (cast.slice(0,4)).
- GET /movie/{id}/images → array backdrops[]. Se seleccionan hasta 5 backdrops ordenados de menor a mayor popularidad (más ambiguos primero).

Flujo de la partida una vez cargados los datos:

- Se muestra backdrops[0] (el más ambiguo) y el campo de búsqueda con autocompletado.
- El jugador escribe en el campo. Con 3 caracteres se lanza GET /search/movie?query=...&language=es-ES y aparece lista desplegable de sugerencias.
- El jugador selecciona un título de la lista. El sistema compara el ID de TMDB de la selección con el ID de la película activa.

- Si coinciden: acierto. Puntuación = $(6 - \text{intentos_usados}) \times 100$. Se guarda en Firestore y se ofrece nueva partida.
- Si no coinciden: se descuenta un intento, se muestra `backdrops[intento]` (siguiente backdrop) y se desvela la pista correspondiente al número de fallo.
- Si se agotan los 5 intentos: puntuación 0, se muestra la respuesta correcta con todos los metadatos y se ofrece nueva partida.

Backdrops progresivos — orden de visualización:

Inicio → `backdrops[0]` (más ambiguo)

Fallo 1 → `backdrops[1]` + pista: año de estreno

Fallo 2 → `backdrops[2]` + pista: director

Fallo 3 → `backdrops[3]` + pista: género

Fallo 4 → `backdrops[4]` + pista: reparto principal

Si la película tiene menos de 5 backdrops se repite el último disponible.

Servicio TMDb en TypeScript:

```
const BASE = 'https://api.themoviedb.org/3';
```

```
const LANG = 'language=es-ES';
```

```
const KEY = `api_key=${process.env.TMDB_API_KEY}`;
```

```
export const getMovie = (id: number) => fetch(`${BASE}/movie/${id}?${KEY}&${LANG}`);
```

```
export const getCredits = (id: number) => fetch(`${BASE}/movie/${id}/credits?${KEY}&${LANG}`);
```

```
export const getImages = (id: number) => fetch(`${BASE}/movie/${id}/images?${KEY}`);
```

```
export const searchMovie = (q: string) =>
  fetch(`${BASE}/search/movie?${KEY}&${LANG}&query=${q}`);
```

4.4. Campo de respuesta con autocompletado

En lugar de un campo de texto libre con normalización, la validación se realiza por ID de TMDb. Esto elimina cualquier problema de acentos, mayúsculas o traducciones parciales.

- El jugador escribe en el campo de búsqueda.
- Cuando hay 3 o más caracteres, se lanza una petición a `/search/movie` con el texto introducido.
- Los resultados aparecen como lista desplegable debajo del campo, mostrando título en español y año.
- Al seleccionar un título de la lista, el sistema extrae su ID de TMDb y lo compara con el ID de la película activa.
- Si los IDs coinciden: acierto. Si no: fallo.

Esta aproximación tiene la ventaja adicional de que el jugador ve los títulos en español sugeridos por TMDb, por lo que la experiencia es completamente en castellano sin necesidad de conocer el título original en inglés.

4.5. Gestión del estado

El estado se gestiona con hooks de React. No se usa Redux ni ninguna librería de estado global porque la complejidad del proyecto no lo justifica. La sesión se gestiona de forma diferente según la plataforma: en web, Firebase Auth persiste la sesión automáticamente mediante localStorage del navegador, detectándola al arrancar mediante onAuthStateChanged. En Android, dado que Firebase 12 no expone getReactNativePersistence de forma compatible con el motor Hermes, la persistencia se implementa manualmente: tras cada login exitoso, las credenciales se guardan en AsyncStorage mediante FirebaseAuthRepository. Al arrancar, onAuthStateChanged detecta que no hay sesión activa y recupera las credenciales guardadas para realizar un login silencioso automático. Al cerrar sesión, las credenciales se eliminan de AsyncStorage antes de llamar a signOut, evitando que onAuthStateChanged intente restaurar la sesión involuntariamente.

Estado	Tipo	Descripción
currentMovie	Object null	Datos de la película activa obtenidos de TMDB en español.
backdrops	string[]	Array de hasta 5 file_path de backdrops ordenados de ambiguo a reconocible.
currentBackdrop	number (0-4)	Índice del backdrop actualmente visible.
attemptsLeft	number (0-5)	Intentos restantes en la partida actual.
hintsVisible	string[]	Pistas de metadatos ya desveladas en la partida activa.
sessionMovies	number[]	IDs de películas ya mostradas en la sesión. Persiste en AsyncStorage.
playerAlias	string	Alias del jugador activo. Recuperado de AsyncStorage al iniciar.
score	number	Puntuación acumulada en la sesión actual.
searchResults	Movie[]	Resultados del autocompletado. Se vacía al seleccionar o cancelar.

4.6. Despliegue

La aplicación se distribuye en dos formatos: versión web accesible desde cualquier navegador y APK instalable en Android.

4.6.1. Versión web — GitHub Pages

La versión web se genera y despliega mediante dos scripts definidos en package.json. Para que las rutas de los assets funcionen correctamente bajo el subpath del repositorio, se configura la propiedad baseUrl con el valor /TFG-SergioGonzalez/ en el fichero app.json. El script predeploy ejecuta `npx expo export --platform web --output-dir dist`, que exporta la aplicación como archivos estáticos en la carpeta dist, seguido de `node fix-paths.js`, un script auxiliar que corrige las rutas absolutas generadas por Expo en el index.html (sustituyendo /_expo/ por /TFG-SergioGonzalez/_expo/) para que los assets se resuelvan correctamente bajo el subpath de GitHub Pages. A continuación, el script deploy ejecuta `gh-pages -d dist -t`, que publica el contenido de dist en la rama gh-pages del repositorio. GitHub Pages sirve automáticamente dicha rama como sitio estático. Las variables de entorno (EXPO_PUBLIC_TMDB_API_KEY) se incluyen en el archivo .env local y no se exponen en el repositorio, aunque sí quedan embebidas en el bundle JavaScript compilado; la solución definitiva a este problema se describe en el apartado de trabajo futuro. El proceso completo se lanza con el comando `npm run deploy`.

La URL pública de la aplicación es: <https://sergiognlz.github.io/TFG-SergioGonzalez/>

4.6.2. APK Android — EAS Build

El APK de Android se genera mediante EAS Build (Expo Application Services), el servicio de compilación en la nube de Expo. El perfil preview, configurado en eas.json con `distribution: internal` y `buildType: apk`, genera un APK de distribución interna directa (sin necesidad de Google Play) usando el comando `eas build -p android --profile preview`. La configuración `distribution: internal` habilita en el panel de Expo una URL de descarga directa y un código QR para facilitar la distribución a testers. Las credenciales de firma (keystore) se gestionan en los servidores de Expo. El APK resultante se instala directamente en dispositivos Android habilitando la instalación desde fuentes desconocidas.

5. Plan de pruebas

El plan de pruebas define los casos necesarios para verificar que la aplicación cumple los requisitos. Se organiza en cuatro niveles: unitarias, integración, funcionales y no funcionales. En total son 48 casos. El detalle completo se recoge en el Anexo I.

Tipo	Total	PASS	FAIL	PEND
Pruebas unitarias	15	15	0	0
Pruebas de integración	9	9	0	0
Pruebas funcionales (caja negra)	13	13	0	0
Pruebas no funcionales	11	10	0	1
TOTAL	48	47	0	1

6. Conclusiones y trabajo futuro

6.1. Conclusiones

Este proyecto ha permitido trabajar con una combinación de tecnologías que no siempre se ven en detalle durante el ciclo: consumo de APIs REST con autenticación por token, gestión de estado en aplicaciones móviles, servicios en la nube con Firebase y despliegue multiplataforma real desde una única base de código.

Una de las cosas que más ha costado durante la fase de análisis ha sido verificar los campos reales de la API de TMDb. La documentación oficial no siempre es clara sobre qué devuelve cada endpoint, y fue necesario contrastar con ejemplos reales para darse cuenta de que el director y el reparto no están en el endpoint principal de detalles sino en un endpoint de créditos separado. Ese tipo de cosas no se aprenden leyendo la documentación, sino intentándolo.

Durante la exploración de la API también surgieron mejoras que no estaban en el diseño inicial: el autocompletado con `/search/movie` simplifica mucho la validación al comparar IDs en lugar de texto normalizado, y los backdrops progresivos convierten una limitación de la API en una mecánica de juego interesante.

6.2. Objetivos alcanzados

Objetivo	Estado (doc.)	Estado (dev.)
App multiplataforma con React Native y Expo	Completado	Completado
Integración con la API de TMDb en español	Completado	Completado
Backdrops progresivos por partida	Completado	Completado
Campo de respuesta con autocompletado	Completado	Completado
Ranking global con Firebase Firestore	Completado	Completado
Plan de pruebas completo (48 casos)	Completado	Completado

6.3. Trabajo futuro

- Filtros por género o época: permitir al jugador elegir categoría de películas antes de empezar.
- Modo multijugador: partidas en tiempo real contra otros jugadores.
- Soporte para series de televisión usando los endpoints de series de TMDb.
- Mejoras en la accesibilidad: soporte completo para lectores de pantalla y modo de alto contraste.

La API Key de TMDb queda embebida en el bundle del cliente tanto en web como en Android, lo que supone un riesgo si la aplicación tuviera distribución amplia. La solución técnica es migrar las llamadas a Firebase Cloud Functions, manteniendo la clave en el servidor. Esta mejora requiere el plan Blaze de Firebase (pago por uso), lo que queda fuera del alcance de un proyecto académico sin presupuesto.

7. Bibliografía y referencias

Ref.	Fuente
[1]	React Native — Documentación oficial. Meta Open Source. https://reactnative.dev/docs/getting-started
[2]	Expo — Documentación oficial. Expo.dev. https://docs.expo.dev
[3]	The Movie Database (TMDB) — API Reference v3. https://developer.themoviedb.org/reference/intro/getting-started
[4]	Firebase — Documentación de Cloud Firestore. Google LLC. https://firebase.google.com/docs/firestore
[5]	TypeScript — Handbook. Microsoft. https://www.typescriptlang.org/docs/handbook/intro.html
[6]	React — Hooks de estado (useState, useEffect). Meta Open Source. https://react.dev/reference/react/useState
[7]	React Native Async Storage. https://react-native-async-storage.github.io/async-storage/docs/install
[8]	Jest — Testing Framework. https://jestjs.io/docs/getting-started
[9]	React Native Testing Library. https://callstack.github.io/react-native-testing-library
[10]	GitHub Pages — Documentación oficial. https://docs.github.com/pages
[11]	Expo Application Services (EAS) — Build. https://docs.expo.dev/build/introduction
[12]	Ley de Propiedad Intelectual (España). Real Decreto Legislativo 1/1996, de 12 de abril. Art. 32.
[13]	Firebase — Documentación de Firebase Authentication. Google LLC. https://firebase.google.com/docs/auth

ANEXO I

Modelo de Datos Conceptual y Plan de Pruebas Completo

A1. Modelo de Datos Conceptual

A1.1. Diagrama Entidad-Relación

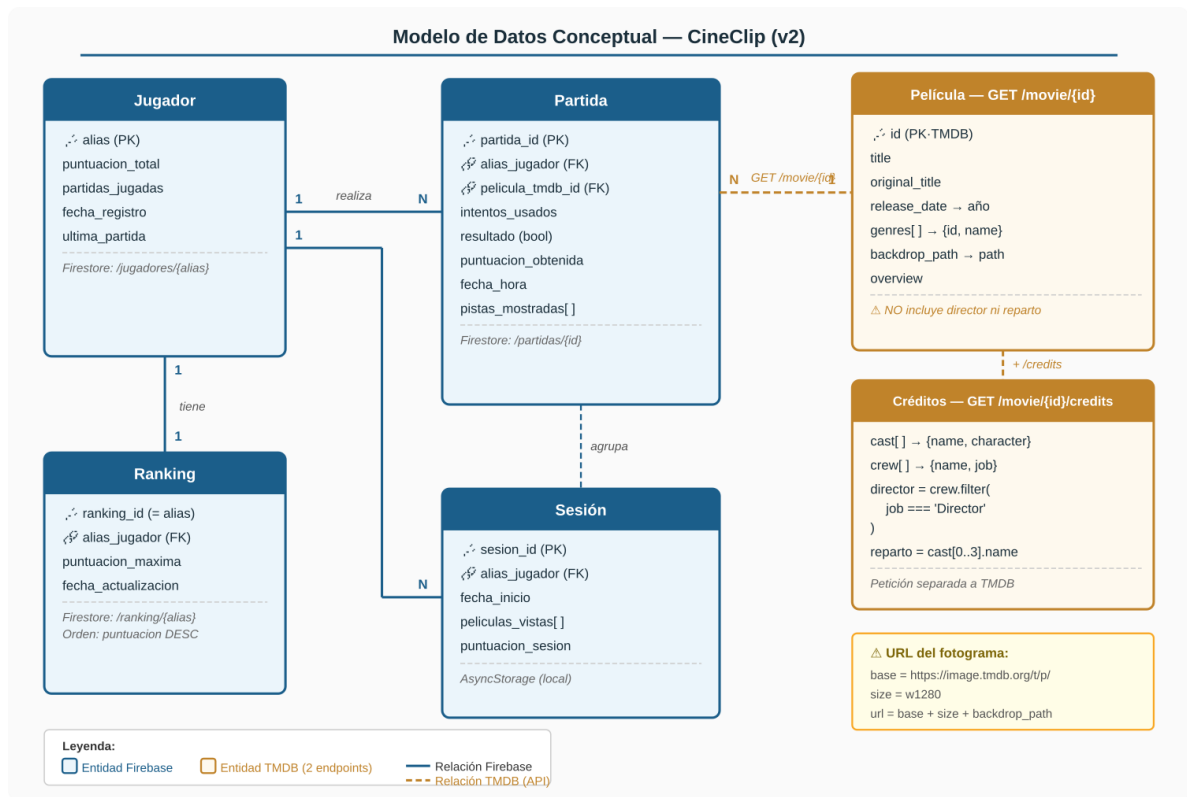


Figura A1. Diagrama Entidad-Relación — CineClip v2

A1.2. Endpoints TMDb por partida

#	Endpoint	Datos obtenidos	Uso en la partida
1	GET /movie/{id}?language=es-ES	title (español), release_date, genres[], backdrop_path	Título validación, año pista 1, género pista 3.
2	GET /movie/{id}/credits?language=es-ES	crew[job==='Director'].name, cast[0..3].name	Director pista 2, reparto pista 4.
3	GET /movie/{id}/images	backdrops[].file_path (hasta 5)	Backdrops progresivos por fallo.
4	GET /search/movie?query=...&language=es-ES	results[].id, results[].title, results[].year	Autocompletado del campo de respuesta.

URL de imagen:

https://image.tmdb.org/t/p/w1280 + file_path

Ejemplo: https://image.tmdb.org/t/p/w1280/c6OLXfKAK5BKeR6broC8pYiCquX.jpg

A1.3. Entidades persistentes en Firestore

Atributo	Tipo	Restricción	Descripción
Jugador — /jugadores/{alias}			
alias	String	PK, 3-20 car.	Identificador único del jugador.
puntuacion_total	Integer	≥ 0	Puntuación acumulada en todas las partidas.
partidas_jugadas	Integer	≥ 0	Total de partidas completadas.
fecha_registro	Timestamp	Not null	Fecha de creación del alias.
ultima_partida	Timestamp	Nullable	Fecha de la última partida jugada.
Partida — /partidas/{id} (reservado para versiones futuras)			
partida_id	String	PK, UUID	Identificador único de la partida.
alias_jugador	String	FK → Jugador	Alias del jugador.
pelicula_tmdb_id	Integer	FK → TMDB	ID de la película en TMDB.
intentos_usados	Integer	1-5	Número de intentos utilizados.
resultado	Boolean	Not null	true = acierto, false = agotó intentos.
puntuacion_obtenida	Integer	≥ 0	(6 - intentos) × 100 si acierto; 0 si fallo.
fecha_hora	Timestamp	Not null	Fecha y hora de finalización.
pistas_mostradas	Array<String>	Max 4	Pistas de metadatos desveladas.
Ranking — /ranking/{alias}			
ranking_id	String	PK = alias	Coincide con el alias del jugador.
puntuacion_maxima	Integer	≥ 0	Puntuación más alta. Consulta DESC.
fecha_actualizacion	Timestamp	Not null	Última actualización del registro.

A2. Plan de Pruebas Completo

A2.1. Pruebas unitarias (PU-01 a PU-15)

ID	Función	Entrada	Resultado esperado	RF	Resultado
PU-01	compareByTmdbId()	id=550, activeId=550	true	RF04	PASS
PU-02	compareByTmdbId()	id=27205, activeId=550	false	RF04	PASS
PU-03	calculateScore()	intentos=1, resultado=true	500 puntos	RF08	PASS
PU-04	calculateScore()	intentos=5, resultado=true	100 puntos	RF08	PASS
PU-05	calculateScore()	intentos=5, resultado=false	0 puntos	RF08	PASS
PU-06	getHint()	fallo=1, pelicula={año:2010}	'Año: 2010'	RF06	PASS
PU-07	getHint()	fallo=2, pelicula={director:'Nolan'}	'Director: Nolan'	RF06	PASS
PU-08	getHint()	fallo=3, pelicula={genero:'Acción'}	'Género: Acción'	RF06	PASS
PU-09	getHint()	fallo=4, pelicula={reparto:'Pitt,...'}	'Reparto: Pitt,...'	RF06	PASS
PU-10	getHint()	fallo=2, director=null	pista omitida sin error	RF06	PASS
PU-11	selectBackdrop()	backdrops=5, fallo=0	backdrops[0]	RF06	PASS
PU-12	selectBackdrop()	backdrops=5, fallo=4	backdrops[4]	RF06	PASS
PU-13	selectBackdrop()	backdrops=2, fallo=4	backdrops[1] (último disponible)	RF06	PASS
PU-14	validateAlias()	'Jugador1'	true	RF01	PASS
PU-15	validateAlias()	"	false + mensaje error	RF01	PASS

A2.2. Pruebas de integración (PI-01 a PI-09)

ID	Integración	Acción	Resultado esperado	RF	Resultado
PI-01	App → TMDB	GET /movie/{id}?language=es-ES	Título en español, release_date, genres[] en < 2s	RF02	PASS

PI-02	App → TMDB	GET /movie/{id}/credits	crew[] y cast[] correctos	RF06	PASS
PI-03	App → TMDB	GET /movie/{id}/images	Array backdrops[] con al menos 1 elemento	RF02	PASS
PI-04	App → TMDB	GET /search/movie?query=inc	Resultados en español con id, title, year	RF03	PASS
PI-05	App → TMDB	Sin conexión	Mensaje de error claro, app no se rompe	RF02	PASS
PI-06	App → Firestore	Guardar puntuación	Documento en /partidas y /ranking actualizados	RF08	PASS
PI-07	App → Firestore	Leer ranking	Lista ordenada por puntuacion_maxima DESC	RF09	PASS
PI-08	App → AsyncStorage	Guardar sesión	Sesión y películas de sesión persisten tras cerrar app	RF02	PASS
PI-09	App → AsyncStorage	Relanzar app	Sesión recuperada correctamente	RF02	PASS

A2.3. Pruebas funcionales — Caja Negra (PF-01 a PF-13)

ID	Caso de uso	Pasos	Resultado esperado	RF	Resultado
PF-01	UC1: Registrar alias	Introducir 'TestUser' y confirmar	Alias aceptado. Juego inicia.	RF01	PASS
PF-02	UC1: Alias vacío	Dejar campo vacío y confirmar	Error: campo obligatorio.	RF01	PASS
PF-03	UC2-UC3: Iniciar partida	Pulsar Nueva Partida	Backdrop[0] visible. Intentos = 5.	RF02	PASS
PF-04	UC4: Autocompletado	Escribir 'Incep' en campo	Lista con 'Inception' y otras sugerencias en español.	RF03	PASS
PF-05	UC4: Respuesta correcta	Seleccionar película correcta	Pantalla de acierto con metadatos en español.	RF04	PASS
PF-06	UC4-5: Respuesta incorrecta	Seleccionar película incorrecta	Intentos = 4. Backdrop[1] visible. Pista año visible.	RF04,RF06	PASS
PF-07	UC5: Backdrops progresivos	Fallar 4 veces	Backdrop cambia en cada fallo. Pistas acumulativas.	RF06	PASS

PF-08	UC4-6: Agotar intentos	Fallar 5 veces	Resultado fallo con respuesta correcta y metadatos.	RF05,RF07	PASS
PF-09	UC7: Guardar puntuación	Finalizar con acierto	Puntuación guardada. Confirmación visible.	RF08	PASS
PF-10	UC8: Ver ranking	Acceder a sección Ranking	Lista por puntuación descendente en español.	RF09	PASS
PF-11	UC9: Nueva partida	Pulsar Nueva Partida tras resultado	Estado reiniciado. Película diferente.	RF02	PASS
PF-12	UC3: Error de imagen	Forzar URL de backdrop inválida	Placeholder visible. Partida no se interrumpe.	RF02	PASS
PF-13	UC4: Película con pocos backdrops	Película con 2 backdrops, fallar 4 veces	Se repite backdrop[1] en fallos 3 y 4 sin errores.	RF06	PASS

A2.4. Pruebas no funcionales (PNF-01 a PNF-11)

ID	Categoría	Prueba	Criterio de aceptación	RNF	Resultado
PNF-01	Rendimiento	Carga de backdrop	Backdrop visible en < 2s en 4G.	RNF01	PASS
PNF-02	Rendimiento	Autocompletado	Sugerencias visibles en < 1s tras 3 caracteres.	RNF01	PASS
PNF-03	Compatibilidad	Android 8+	Funciona en emulador API 26.	RNF07	PEND
PNF-04	Compatibilidad	Web móvil	Funciona en navegador móvil (Chrome Android).	RNF07	PASS
PNF-05	Compatibilidad	Web Chrome	Carga y funciona en Chrome actual.	RNF07	PASS
PNF-06	Compatibilidad	Responsive	Adapta a 360px, 768px y 1280px.	RNF08	PASS
PNF-07	Usabilidad	Sin manual	Partida completada en < 3 min por usuario nuevo.	RNF05	PASS
PNF-08	Usabilidad	Máx. 3 clics	Acciones principales en ≤ 3 interacciones.	RNF06	PASS
PNF-09	Seguridad	HTTPS	Todas las peticiones van cifradas.	RNF03	PASS
PNF-10	Seguridad	Reglas Firebase	No escritura directa en Firestore sin la app.	RNF04	PASS
PNF-11	Accesibilidad	Contraste	Ratio texto/fondo ≥ 4.5:1 (WCAG AA).	RNF10	PASS

A2.5. Resumen de seguimiento

Tipo	Total	PASS	FAIL	PEND
Pruebas unitarias	15	15	0	0
Pruebas de integración	9	9	0	0
Pruebas funcionales (caja negra)	13	13	0	0
Pruebas no funcionales	11	10	0	1
TOTAL	48	47	0	1